

DFS of Graph

Solved

Difficulty: **Easy** Accuracy: **63.07%** Submissions: **418K+** Points: **2** Average Time: **5m**

Given a **connected undirected graph** containing **V** vertices represented by a 2-d adjacency list **adj[][]**, where each **adj[i]** represents the list of vertices connected to vertex **i**. Perform a **Depth First Search (DFS)** traversal starting from vertex **0**, visiting vertices from left to right as per the given adjacency list, and return a list containing the DFS traversal of the graph.

Note: Do traverse in the **same order** as they are in the given **adjacency list**.

Examples:

Input: `adj[][] = [[2, 3, 1], [0], [0, 4], [0], [2]]`

Output: `[0, 2, 4, 3, 1]`

Explanation: Starting from 0, the DFS traversal proceeds as follows:

Visit 0 → Output: 0

Visit 2 (the first neighbor of 0) → Output: 0, 2

Visit 4 (the first neighbor of 2) → Output: 0, 2, 4

Backtrack to 2, then backtrack to 0, and visit 3 → Output: 0, 2, 4, 3

Finally, backtrack to 0 and visit 1 → Final Output: 0, 2, 4, 3, 1

Input: `adj[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]`

Output: `[0, 1, 2, 3, 4]`

Explanation: Starting from 0, the DFS traversal proceeds as follows:

Visit 0 → Output: 0

Visit 1 (the first neighbor of 0) → Output: 0, 1

Visit 2 (the first neighbor of 1) → Output: 0, 1, 2

Visit 3 (the first neighbor of 2) → Output: 0, 1, 2, 3

Backtrack to 2 and visit 4 → Final Output: 0, 1, 2, 3, 4